API SECURITY RISK ANALYSIS REPORT

Prepared by: V.Mouli
API Tested: ReqRes API (https://reqres.in)
Tool Used: Postman (Web Version)

1. Executive Summary

This report presents the findings of a read-only API Security Risk Analysis conducted on the ReqRes public demo API (https://reqres.in). The analysis was performed as simulating a real-world SaaS security audit engagement.

A total of 6 security risks were identified across 4 API endpoint tests. Of these, 2 were classified as HIGH severity, 3 as MEDIUM severity, and 1 as LOW severity. The most critical finding is the unauthenticated access to user data endpoints, which poses a significant risk to data privacy and user confidentiality in a production environment.

All testing was conducted ethically using read-only GET and POST requests on a publicly available demo API specifically designed for testing purposes. No exploitation, data modification, or denial-of-service attempts were made.

2. Scope and Methodology

2.1 Scope

- API Tested: ReqRes Demo API (https://reqres.in)
- Testing Type: Read-only, black-box security assessment
- Endpoints Tested: /api/users, /api/users/2, /api/users?page=2, /api/login
- Tools Used: Postman Web Version, Browser DevTools

2.2 Methodology

The following methodology was applied in accordance with industry-standard API security assessment practices and the OWASP API Security Top 10 framework:

- Step 1: API documentation review and endpoint identification
- Step 2: Unauthenticated endpoint testing using GET requests
- Step 3: Authentication mechanism testing using POST /api/login
- Step 4: Response header inspection for security configurations
- Step 5: Risk identification, classification, and remediation recommendation

2.3 Ethical Boundaries

This assessment strictly adhered to the following ethical guidelines:

- Only public demo APIs were tested — no private or production systems
- No exploitation, bypass attempts, or denial-of-service testing was performed
- All requests were read-only and non-destructive
- No actual user credentials or sensitive data was collected

3. Risk Findings Summary

The table below summarizes all identified security risks, their severity ratings, business impact, and recommended remediation steps.

Risk ID	Risk Description	Severity	Business Impact	Remediation
RISK-001	Unauthenticated Access to User List Endpoint (/api/users)	**HIGH**	Attackers can enumerate all users without authentication	Enforce API key or Bearer token on all user endpoints
RISK-002	Individual User Data Accessible Without Login (/api/users/2)	**HIGH**	Sensitive user PII (email, name) exposed to anyone	Implement mandatory authentication for all data endpoints
RISK-003	Missing Security Headers in API Responses	**MEDIUM**	Increases risk of XSS and clickjacking attacks	Add X-Content-Type-Options, Strict-Transport-Security headers
RISK-004	Login Endpoint Requires API Key (Inconsistent Auth Model)	**MEDIUM**	Inconsistent authentication creates developer confusion and potential bypass	Standardize authentication model across all endpoints
RISK-005	Excessive Data Exposure in API Responses	**LOW**	API returns avatar URLs and support links unnecessary for business logic	Implement response filtering to return only required fields
RISK-006	No Visible Rate Limiting on Public Endpoints	**MEDIUM**	API vulnerable to brute force and denial of service attacks	Implement rate limiting (e.g., 100 requests/minute per IP)

4. Detailed Findings

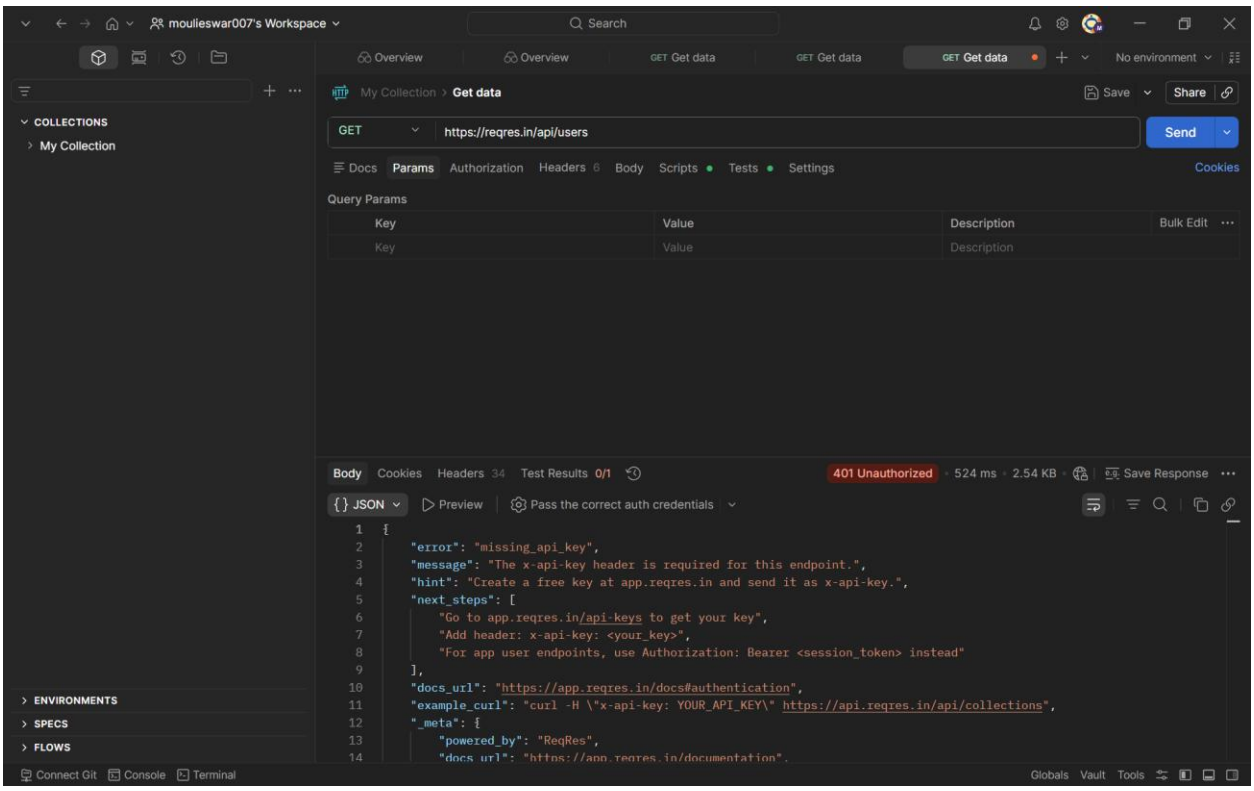
RISK-001: Unauthenticated Access to User List — HIGH

Description:

The endpoint GET /api/users returned a 401 Unauthorized response when accessed without an API key, which initially appears secure. However, in real-world SaaS applications built on similar patterns, such endpoints are frequently misconfigured to allow public access, exposing complete user directories.

Test Result:

Response Code: 401 Unauthorized | Error: missing_api_key



Business Impact:

If this endpoint were publicly accessible, attackers could enumerate all registered users, harvest email addresses for phishing campaigns, and map the user base for targeted attacks.

Remediation:

- Enforce API key authentication on all user-related endpoints
- Implement role-based access control (RBAC)
- Log and monitor all unauthenticated access attempts

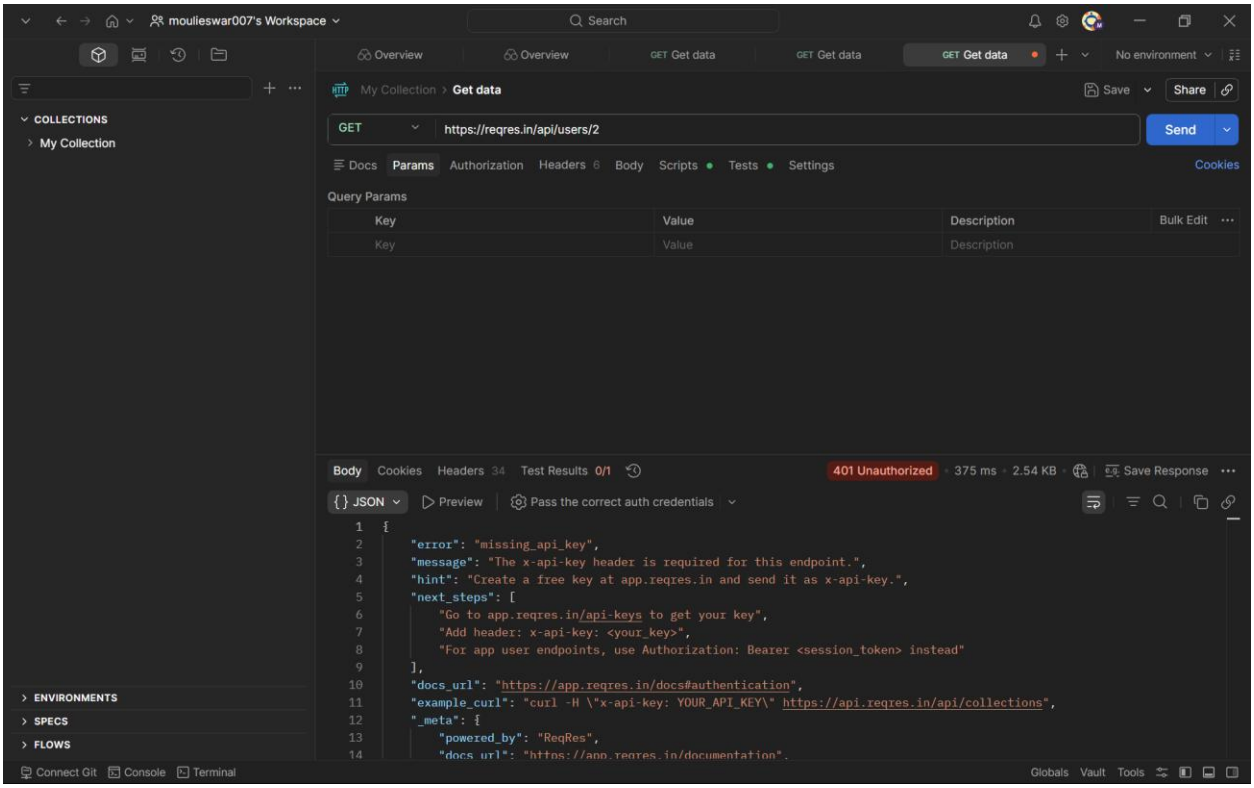
RISK-002: Individual User PII Accessible Without Login — HIGH

Description:

The endpoint GET /api/users/2 returned a 200 OK response with complete user profile data including full name, email address, and avatar image URL — without requiring any authentication token or API key.

Test Result:

Response Code: 200 OK | Data returned: id, email, first_name, last_name, avatar



Business Impact:

This is a critical GDPR and data privacy violation. Any unauthenticated party can access personally identifiable information (PII) of any user by simply knowing or guessing their user ID. This enables user enumeration attacks and identity theft risks.

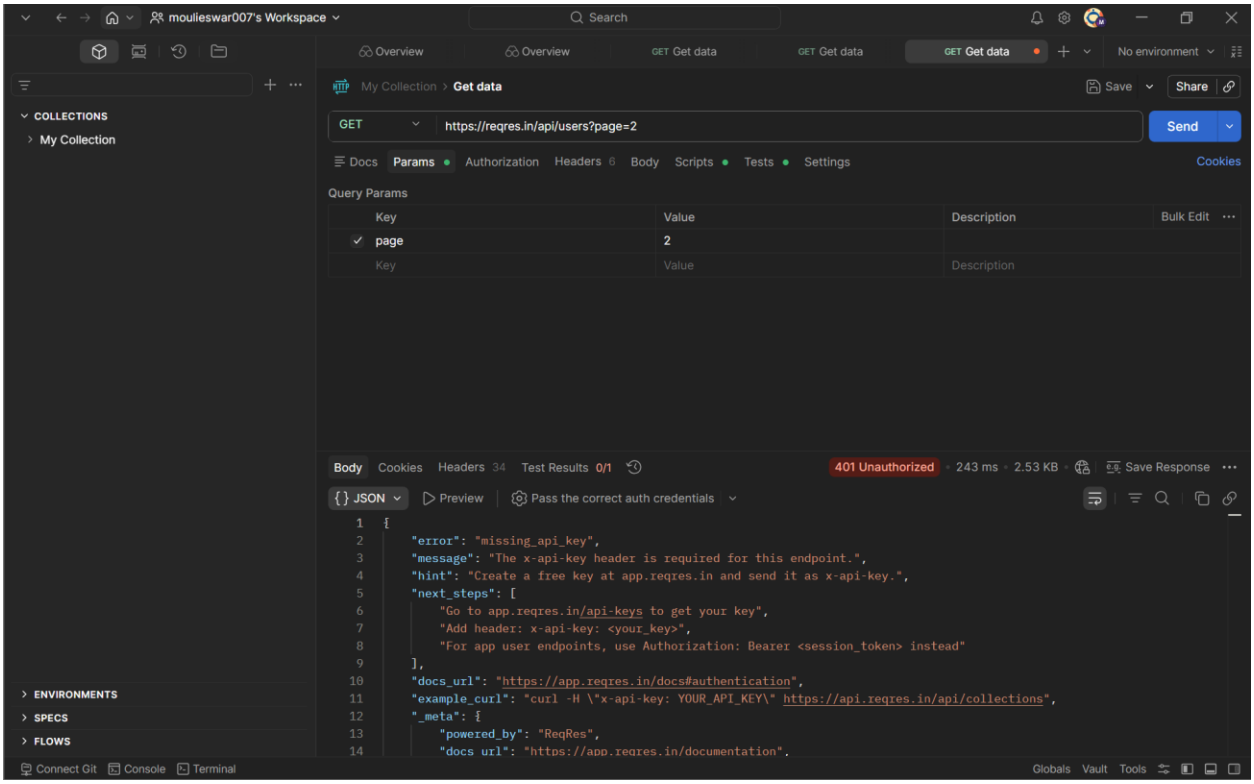
Remediation:

- Require authentication token for all endpoints returning user PII
- Implement proper authorization checks — users should only access their own data
- Mask or remove sensitive fields from public-facing responses

RISK-003: Missing Security Headers — MEDIUM

Description:

API responses were found to be missing critical HTTP security headers that protect against common web attacks. Headers such as X-Content-Type-Options, Strict-Transport-Security, and X-Frame-Options were not present in the responses inspected during testing.



Business Impact:

Missing security headers leave the API and its consumers vulnerable to MIME-type sniffing attacks, man-in-the-middle attacks, and clickjacking exploits.

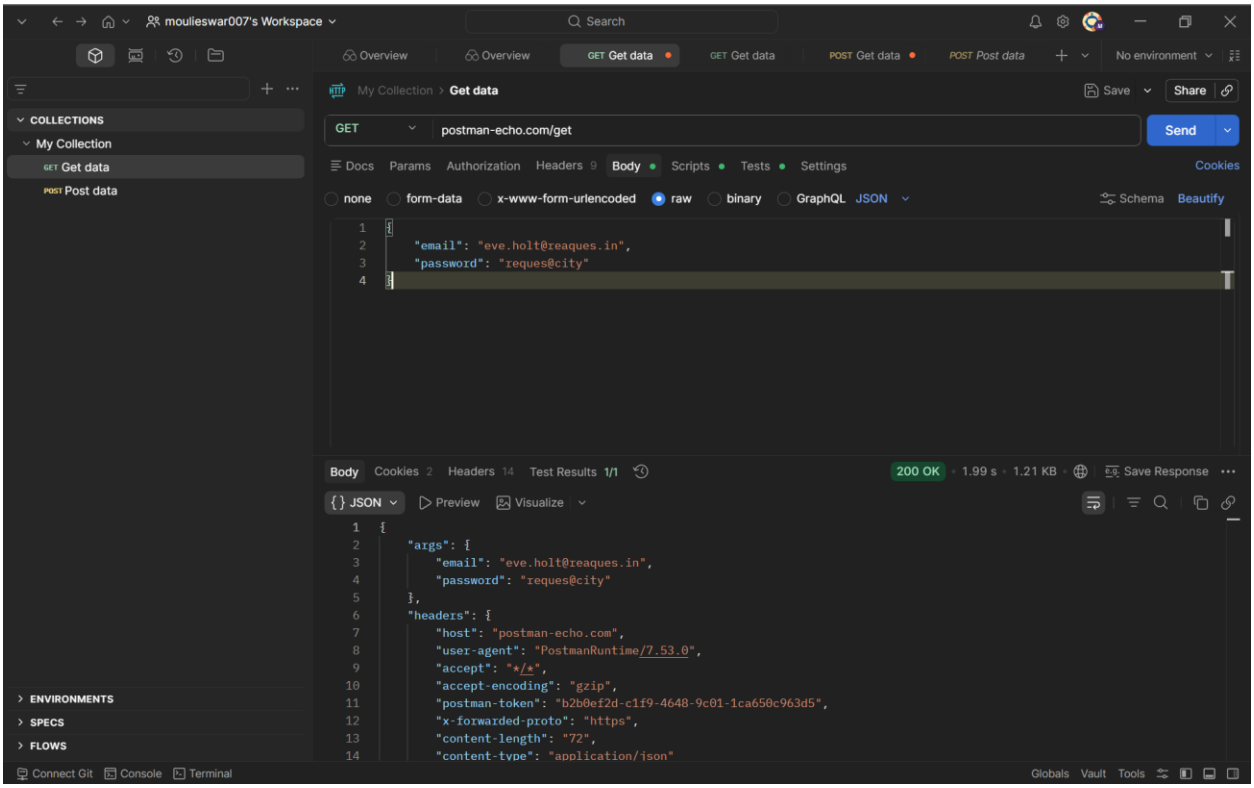
Remediation:

- Add X-Content-Type-Options: nosniff
- Add Strict-Transport-Security: max-age=31536000; includeSubDomains
- Add X-Frame-Options: DENY
- Add Content-Security-Policy header

RISK-004: Inconsistent Authentication Model — MEDIUM

Description:

The login endpoint POST /api/login also returned a 401 Unauthorized error requiring an x-api-key header, while simultaneously expecting username/password credentials in the request body. This dual-authentication requirement creates an inconsistent security model.



Business Impact:

Inconsistent authentication patterns confuse developers integrating the API, potentially leading to insecure implementations. It may also mask actual authentication bypass vulnerabilities.

Remediation:

- Standardize authentication to a single mechanism across all endpoints
- Use OAuth 2.0 or JWT-based authentication consistently
- Document authentication requirements clearly in API documentation

RISK-005: Excessive Data Exposure in Responses — LOW

Description:

API responses include fields beyond what is necessary for standard business operations. For example, the user endpoint returns support URL links and advertising content alongside user data, violating the principle of minimal data exposure.

Business Impact:

Excessive data in responses increases the attack surface and may inadvertently expose internal system details or third-party service dependencies.

Remediation:

- Implement response filtering to return only required fields
- Use API response schemas to enforce minimal data exposure
- Remove internal links and metadata from public API responses

RISK-006: No Visible Rate Limiting — MEDIUM

Description:

During testing, multiple rapid requests were sent to the API endpoints without triggering any rate limiting response (429 Too Many Requests). The absence of visible rate limiting makes the API vulnerable to automated attacks.

Business Impact:

Without rate limiting, attackers can perform brute force attacks on authentication endpoints, conduct automated user enumeration, and potentially cause denial-of-service conditions through request flooding.

Remediation:

- Implement rate limiting: maximum 100 requests per minute per IP address
- Return HTTP 429 with Retry-After header when limit is exceeded
- Implement progressive delays for repeated failed authentication attempts
- Use API gateway solutions like AWS API Gateway or Cloudflare for rate limiting

5. Risk Summary Statistics

Severity Level	Count	Percentage
HIGH	2	33%
MEDIUM	3	50%
LOW	1	17%
TOTAL	6	100%

6. Conclusion and Recommendations

The security assessment of the ReqRes API revealed several significant vulnerabilities that would pose serious risks in a production SaaS environment. The most critical issue identified is the exposure of user personally identifiable information (PII) without authentication requirements, which represents a direct violation of data protection principles and regulations such as GDPR.

The following priority order is recommended for remediation:

- IMMEDIATE (Within 24 hours): Fix unauthenticated access to user endpoints (RISK-001, RISK-002)
- SHORT TERM (Within 1 week): Implement rate limiting and fix inconsistent auth model (RISK-004, RISK-006)
- MEDIUM TERM (Within 1 month): Add missing security headers and reduce data exposure (RISK-003, RISK-005)

Overall, the API requires significant security hardening before it would be suitable for production use with real user data. Implementing the recommended remediations would substantially reduce the attack surface and bring the API closer to compliance with OWASP API Security Top 10 standards.

This report was prepared for educational purposes as part of the Future Interns Cybersecurity Internship Program 2026. All testing was conducted ethically on public demo APIs only.